

MÓDULO 05: Introducción a la Programación Orientada a Objetos (POO) en Ciencia Abierta

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora

Módulo: 05 - Object-Oriented Programming: Classes, Objects, Attributes & Scientific Repositories

1. Objetivos de Aprendizaje

- **Fundamentos de POO:** Comprender los conceptos de Clase (molde/plantilla) e Instancia/Objeto (entidad concreta en memoria).
 - **Atributos y Métodos:** Definir atributos de estado y métodos de comportamiento dentro de clases de Python.
 - Atributos: ¿Qué propiedades tiene el objeto?
 - Métodos: ¿Qué puede hacer el objeto?
 - **Modelado de Dominio Científico:** Diseñar las clases `Persona` (autores científicos), `Paper` (publicación científica) y `BibliotecaCientifica` (repositorio de acceso abierto).
 - **Entender el Concepto de Hash (`__hash__` / `__eq__`):** Comprender cómo funciona la huella digital numérica para la indexación rápida en conjuntos `set()` y la detección de artículos duplicados por DOI.
 - **Conexión con Pandas/NumPy:** Entender por qué en Python y en la Ciencia de Datos "todo es un objeto" (atributos de estado como `df.shape` frente a métodos como `df.describe()`).
-

2. ¿Por qué POO en Ciencia Abierta y Análisis de Datos?

En la programación estructurada organizamos el código en variables y funciones independientes. En la **Programación Orientada a Objetos (POO)**, agrupamos los **datos (atributos)** y las **operaciones (métodos)** que pertenecen a una misma entidad conceptual.

En Python, **todo es un objeto**:

- Una cadena de texto (`str`) es un objeto con métodos como `.strip()` o `.upper()`.
- Un `DataFrame` de Pandas es un objeto con atributos como `.shape` y métodos como `.mean()`.

Conceptos Clave:

- **Clase (`class`)**: La plantilla o plano arquitectónico (ejemplo: la definición general de lo que es un *Paper* científico).
- **Objeto / Instancia**: Una entidad real creada a partir de la clase (ejemplo: un artículo específico publicado en Nature con su DOI concreto).

3. Primera Clase: `Persona` (Autores e Investigadores Científicos)

Comenzamos definiendo una clase sencilla para representar investigadoras e investigadores.

- Método `__init__(self, ...)`: Es el **constructor** de la clase. Se ejecuta automáticamente al crear un nuevo objeto para inicializar sus atributos.
- Parámetro `self`: Representa la propia instancia del objeto que se está creando o manipulando.
- Método `__str__(self)`: Define la representación legible en texto cuando imprimimos el objeto con `print()`.

```
In [ ]: class Persona:
        """Representa a un investigador o autor científico."""

        def __init__(self, nombre, apellidos, filiacion="CSIC"):
            self.nombre = nombre
            self.apellidos = apellidos
            self.filiacion = filiacion

        def nombre_completo(self):
            """Devuelve el nombre en formato bibliográfico (Apellidos, Inicial.)
            return f"{self.apellidos}, {self.nombre[0]}."
```

```

def __str__(self):
    """Representación legible en texto del autor."""
    return f"{self.nombre_completo()} ({self.filiacion})"

# Instanciación de objetos de la clase Persona
autor1 = Persona("Gustavo", "Liñán Cembrano", "IMSE-CNM (CSIC – Univ. de Sev)
autor2 = Persona("Ada", "Lovelace", "University of London")
autor3 = Persona("Santiago", "Ramón y Cajal", "Instituto Cajal (CSIC)")

print("=== INSTANCIAS CREADAS DE LA CLASE PERSONA ===")
print(f"Autor 1: {autor1}")
print(f"Autor 2: {autor2}")
print(f"Autor 3: {autor3}")

```

4. Entendiendo el Concepto de Hash (hash()) en Python

Antes de construir la clase `Paper`, es esencial comprender qué es el **Hashing** y por qué es fundamental en las estructuras de datos de Python (como conjuntos `set` y diccionarios `dict`).

💡 ¿Qué es un Hash?

Un **hash** es un algoritmo matemático que transforma cualquier conjunto de datos (como una cadena de texto de un DOI) en un **número entero único que actúa como su "huella dactilar digital"**.

```

doi_ejemplo = "10.1038/s41586-026-0001-a"
print(hash(doi_ejemplo)) # Devuelve un entero único que representa
a esa cadena

```

¿Por qué lo usan los Conjuntos (set) y Diccionarios (dict)?

1. **Búsqueda Ultrarrápida $\mathcal{O}(1)$:** Cuando buscas si un artículo está en un conjunto de 1.000.000 de registros, Python no compara el título uno a uno. Calcula el `hash(doi)` y va directamente a la posición de memoria (el "cajón") donde debe estar.
2. **Identidad e Inmutabilidad (*Hashable*):** Un objeto se considera *hashable* si su huella digital no cambia a lo largo de su vida útil.
3. **Detección de Duplicados:** Si dos objetos producen el mismo hash y la comparación `__eq__` devuelve `True`, Python sabe inmediatamente que son el mismo objeto y el `set()` descarta el duplicado.

```

In [ ]: # Demostración del funcionamiento nativo de hash() en Python
doi1 = "10.1038/s41586-026-0001-a"

```

```
doi2 = "10.1038/s41586-026-0001-a" # Idéntico a doi1
doi3 = "10.1098/rstl.1843.0018"    # Diferente

print("=== DEMOSTRACIÓN DEL CÁLCULO DE HASH ===")
print(f"Hash DOI 1: {hash(doi1)}")
print(f"Hash DOI 2: {hash(doi2)}")
print(f"Hash DOI 3: {hash(doi3)}")

print("")
print(f"¿Tienen el mismo hash DOI 1 y DOI 2? {hash(doi1) == hash(doi2)}")
print(f"¿Tienen el mismo hash DOI 1 y DOI 3? {hash(doi1) == hash(doi3)}")
```

5. Composición de Objetos y Sobrescritura de `__hash__` / `__eq__`: La Clase `Paper`

Una ventaja clave de la POO es la **composición**: los atributos de un objeto pueden ser colecciones de otros objetos. Un artículo científico (`Paper`) contiene una lista o conjunto de objetos `Persona` como autores.

Implementación de `__eq__` y `__hash__` en `Paper`:

- `__eq__(self, other)` : Le dice a Python que dos objetos `Paper` son lógicamente iguales si comparten el mismo `doi`.
- `__hash__(self)` : Le indica a Python que utilice el entero `hash(self.doi)` como la huella digital del objeto. De este modo, un conjunto `set()` sabrá identificar artículos duplicados por su DOI.

```
In [ ]: class Paper:
        """Representa una publicación o artículo científico en un entorno de Cien

        def __init__(self, titulo, autores:set[Persona], tipo="Artículo", publicacion:publicacion, volumen:volumen, paginas:paginas, año:año, doi:doi):
            self.titulo = titulo
            self.autores = autores # Lista de objetos de tipo Persona
            self.tipo = tipo
            self.publicacion = publicacion
            self.volumen = volumen
            self.paginas = paginas
            self.año = año
            self.doi = doi.strip().lower()

        def cita_apo(self):
            """Genera la cita bibliográfica en formato APA estándar."""
            nombres_autores = "; ".join([a.nombre_completo() for a in self.autores])
            return f"{nombres_autores} ({self.año}). {self.titulo}. {self.publicacion} {self.volumen} {self.paginas} {self.año}"

        def __eq__(self, other):
            """Dos papers se consideran iguales si tienen el mismo DOI."""
            if isinstance(other, Paper):
```

```

        return self.doi == other.doi
    return False

    def __hash__(self):
        """Devuelve el hash del DOI para permitir el almacenamiento único en
        return hash(self.doi)

    def __str__(self):
        return f"[{self.tipo}] '{self.titulo}' ({self.año}) - DOI: {self.doi}"

# Crear instancias de Paper usando las instancias previas de Persona

paper1 = Paper(
    titulo="AI-Driven Open Science Infrastructures in CSIC",
    autores=[autor1, autor3],
    tipo="Artículo",
    publicacion="Nature Open Science",
    volumen=42,
    paginas="101-115",
    año=2026,
    doi="10.1038/s41586-026-0001-a"
)

paper2 = Paper(
    titulo="Analytical Engine Algorithms and Scientific Computing",
    autores=[autor2],
    tipo="Revisión",
    publicacion="Philosophical Transactions",
    volumen=12,
    paginas="45-68",
    año=1843,
    doi="10.1098/rstl.1843.0018"
)

print("=== GENERACIÓN AUTOMÁTICA DE CITAS BIBLIOGRÁFICAS ===")
print("Paper 1 (Cita APA):")
print(f"    {paper1.cita_apa()}")
print("")
print("Paper 2 (Cita APA):")
print(f"    {paper2.cita_apa()}")
print("")
print(f"Hash del objeto paper1: {hash(paper1)}")
print(f"Hash del objeto paper2: {hash(paper2)}")

```

6. Colección y Repositorio: La Clase BibliotecaCientifica

Diseñamos una clase encargada de gestionar el repositorio institucional. Utiliza un `set()` interno para almacenar los artículos, **impidiendo de forma nativa la adición de artículos duplicados** que tengan el mismo DOI gracias a nuestro método `__hash__()`.

```
In [18]: class BibliotecaCientifica:
    """Gestiona de manera FICTICIA!!! Los softwares reales son mucho más con
    un repositorio o biblioteca digital de artículos científicos sin duplica

    def __init__(self, nombre_repositorio="GUS-LIBRARY Open Access", adminis
    self.nombre_repositorio = nombre_repositorio
    self.papers = set() # Colección set para garantizar unicidad por DOI
    self.administrador = administrador # Objeto de tipo Persona que adm

    def agregar_paper(self, paper):
        """Agrega un nuevo artículo a la biblioteca si no existe previamente
        if paper in self.papers:
            print(f"⚠ El artículo con DOI '{paper.doi}' ya existe en el rep
        else:
            self.papers.add(paper)
            print(f"✓ Artículo añadido con éxito: '{paper.titulo}'")

    def buscar_por_autor(self, apellido):
        """Busca todos los artículos donde alguno de los autores coincida co
        resultados = []
        for paper in self.papers:
            for autor in paper.autores:
                if apellido.lower() in autor.apellidos.lower():
                    resultados.append(paper)
                    break
        return resultados

    def resumen_repositorio(self):
        """Muestra el estado completo del repositorio científico."""
        print("")
        print(f"=== REPOSITORIO: {self.nombre_repositorio} ===")
        if self.administrador:
            print(f"Administrador/a del repositorio: {self.administrador}")
        print(f"Total de artículos indexados: {len(self.papers)}")
        for idx, p in enumerate(self.papers, 1):
            print(f"  {idx}. {p}")
```

```
In [19]: # Creación de la biblioteca e inserción de artículos
admin_csic = Persona("Gustavo", "Liñan", "Mi casa mi templo")
repositorio_csic = BibliotecaCientifica("Repo personal de GustavoLiñan", adm

print("=== GESTIÓN E INSERCIÓN EN EL REPOSITORIO CSIC ===")
repositorio_csic.agregar_paper(paper1)
repositorio_csic.agregar_paper(paper2)

# Intentar insertar un artículo duplicado con el mismo DOI
paper_duplicado = Paper(
    titulo="Título diferente pero mismo DOI que Paper 1",
    autores=[autor1],
    doi="10.1038/s41586-026-0001-a" # Mismo DOI que paper1
)
print("")
print("Intento de adición de un duplicado por DOI:")
repositorio_csic.agregar_paper(paper_duplicado)
```

```
# Visualizar el estado final del repositorio
repositorio_csic.resumen_repositorio()

# Búsqueda temática por autor
print("")
print("=== BÚSQUEDA FILTRADA POR AUTOR ('Liñán') ===")
hallados = repositorio_csic.buscar_por_autor("Liñán")
for item in hallados:
    print(f" - Cita APA: {item.cita_apa()}")
```

=== GESTIÓN E INSERCIÓN EN EL REPOSITORIO CSIC ===

✓ Artículo añadido con éxito: 'AI-Driven Open Science Infrastructures in CSI C'

✓ Artículo añadido con éxito: 'Analytical Engine Algorithms and Scientific Computing'

Intento de adición de un duplicado por DOI:

⚠ El artículo con DOI '10.1038/s41586-026-0001-a' ya existe en el repositorio. Registro rechazado.

=== REPOSITORIO: Repo personal de GustavoLiñan ===

Administrador/a del repositorio: Liñan, G. (Mi casa mi templo)

Total de artículos indexados: 2

1. [Artículo] 'AI-Driven Open Science Infrastructures in CSIC' (2026) – DOI: 10.1038/s41586-026-0001-a

2. [Revisión] 'Analytical Engine Algorithms and Scientific Computing' (1843) – DOI: 10.1098/rstl.1843.0018

=== BÚSQUEDA FILTRADA POR AUTOR ('Liñán') ===

- Cita APA: Liñán Cembrano, G.; Ramón y Cajal, S. (2026). AI-Driven Open Science Infrastructures in CSIC. Nature Open Science, 42, 101–115. <https://doi.org/10.1038/s41586-026-0001-a>

7. Ejercicios Prácticos de Autoevaluación

Ejercicio 1:

Añade un método `es_reciente(self, año_corte=2020)` a la clase `Paper` que devuelva `True` si el artículo fue publicado en el año `año_corte` o posterior, y `False` en caso contrario.

Ejercicio 2:

Añade un método `filtrar_por_tipo(self, tipo_buscado)` a la clase `BibliotecaCientifica` que devuelva una lista con todas las publicaciones cuya propiedad `tipo` coincida con `tipo_buscado` (por ejemplo, `"Artículo"` o `"Revisión"`).

```

In [ ]: # SOLUCIÓN INTEGRADA – MÓDULO 05

# Ampliación del método es_reciente en la clase Paper
def paper_es_reciente(self, año_corte=2020):
    """Devuelve True si la publicación es del año_corte o posterior."""
    return self.año >= año_corte

# Asignación dinámica del método a la clase Paper
Paper.es_reciente = paper_es_reciente

# Ampliación del método filtrar_por_tipo en la clase BibliotecaCientifica
def biblioteca_filtrar_por_tipo(self, tipo_buscado):
    """Filtra y devuelve publicaciones que coincidan con el tipo especificado"""
    return [p for p in self.papers if p.tipo.lower() == tipo_buscado.lower()]

BibliotecaCientifica.filtrar_por_tipo = biblioteca_filtrar_por_tipo

print("=== VERIFICACIÓN DEL SOLUCIONARIO DE P00 ===")
print(f"¿Es reciente paper1 (publicado en {paper1.año}) desde 2020? {paper1.es_reciente}")
print(f"¿Es reciente paper2 (publicado en {paper2.año}) desde 2020? {paper2.es_reciente}")

revisiones = repositorio_csic.filtrar_por_tipo("Revisión")
print("")
print(f"Publicaciones de tipo 'Revisión' encontradas ({len(revisiones)}):")
for rev in revisiones:
    print(f"    - {rev.titulo} ({rev.año})")

```